

Reviewer Pack — SOS Moment Matrix Eigenvalue Threshold for ACC⁰ Lower Bounds

Entry 3654b18c7301 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 04:03:27 UTC

SOS Moment Matrix Eigenvalue Threshold for ACC⁰ Lower Bounds

Entry ID: 3654b18c7301 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 04:03:27 UTC

1. Conjecture

Field A (mathematical branch): SUM_OF_SQUARES_HIERARCHY **Field B** (complexity object): ACC_ZERO_CIRCUIT_COMPLEXITY

Statement:

For an explicit function $f: \{0,1\}^n \rightarrow \{0,1\}$ in P , if the minimal eigenvalue $\lambda_{\min}$ of its degree- d SOS moment matrix satisfies $\lambda_{\min} \geq \Omega(1/\sqrt{n})$, then f cannot be computed by ACC⁰ circuits of size $o(n^{1/4})$ for any constant $d \geq 2$.

Rationale (proposer's reasoning):

The SOS moment matrix eigenvalue captures the 'hardness' of refuting f via low-degree polynomial approximations. If $\lambda_{\min}$ is sufficiently large, it implies the function resists ACC⁰ computation, aligning with Williams' $NEXP \not\subseteq ACC^0$ separation. This property is computable in subexponential time for $n \leq 40$ via numerical linear algebra, avoiding natural proofs barriers by focusing on structural eigenvalue thresholds rather than direct circuit analysis.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 051c7bad7b47eef4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=34.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import sys
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_function(n):
        return lambda x: sum(x[i] * (i + 1) for i in range(n)) % 2

    def degree_2_sos_moment_matrix(f, n):
        M = [[0] * n for _ in range(n)]
        for x in itertools.product([0, 1], repeat=n):
            f_x = f(x)
            for i in range(n):
                for j in range(n):
                    M[i][j] += x[i] * x[j] * (i + 1) * (j + 1)
        return M

    def power_iteration(M, n, max_iter=1000):
        v = [random.random() for _ in range(n)]
        v = [x / sum(v) for x in v]
        for _ in range(max_iter):
            v_next = [sum(M[i][j] * v[j] for j in range(n)) for i in range(n)]
            v_next = [x / sum(v_next) for x in v_next]
            if all(abs(v_next[i] - v[i]) < 1e-6 for i in range(n)):
                break
            v = v_next
        return max(v), min(v)

    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_function(n)
    M = degree_2_sos_moment_matrix(f, n)
    lambda_max, lambda_min = power_iteration(M, n)
```

```

if lambda_min <= 0:
    return {
        "metric_name": "lambda_min",
        "metric_value": lambda_min,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "negative_eigenvalue"
    }

conjecture_holds = lambda_min >= Fraction(1, math.sqrt(n))
counterexample = "" if conjecture_holds else f"lambda_min={lambda_min} < 1/√{n}"

return {
    "metric_name": "lambda_min",
    "metric_value": lambda_min,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else list(range(2, 200, 7))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_40a05773.py", line 73, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_40a05773.py", line 57, in run_trial

```

    conjecture_holds = lambda_min >= Fraction(1, math.sqrt(n))

```

```

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in comparison; support condition not met | next: Fix type error in $\lambda_{\min}$ comparison by converting `math.sqrt(n)` to Fraction or rational type

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	56959
2	preregistration	ollama_remote	qwen3:8b	0	0	27860
3	novelty	ollama_remote	qwen3:8b	0	0	24197
4	novelty	ollama_remote	qwen3:8b	0	0	19403
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16305
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10663
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8965
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9701
9	judge	ollama_remote	qwen3:8b	0	0	19108

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 193163 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3654b18c7301.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3654b18c7301.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3654b18c7301.tar.gz` (if generated)